

Teoría de la Computabilidad

Módulo 10: Tesis de Turing – Church / Introducción a la Reducibilidad

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Procedimiento Efectivo

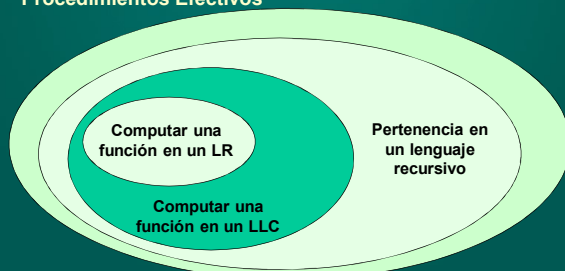
Un **procedimiento efectivo** (p.e.) es un **conjunto de reglas**, destinadas a resolver un problema, escritas en un determinado lenguaje, que son **interpretables y ejecutables**.

Esta definición posee un alto grado de abstracción. Intenta capturar aquellos mecanismos de resolución de algún problema cualquiera...

“**conjunto de reglas**” puede ser un conjunto de instrucciones en Pascal, o en LDA, o en castellano sin ambigüedades o imprecisiones... ..

Procedimiento Efectivo

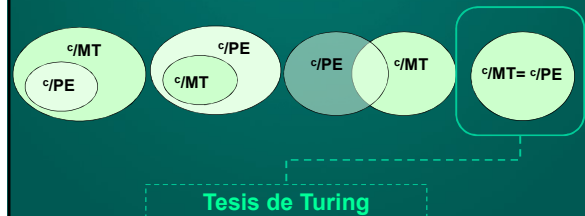
Problemas Solubles por
Procedimientos Efectivos



¿Qué relación existirá entre los problemas solubles por PE y aquellos solubles por Máquinas de Turing?

Procedimiento Efectivo

¿Qué relación existirá entre los problemas solubles por PE y aquellos solubles por Máquinas de Turing?



Turing propuso una tesis (no teorema) que afirma la equivalencia de los dos conceptos: PE (informal) y MT (formal)

La Tesis de Turing-Church

Tengamos en cuenta que:

Si una función f es **computada por una MT T** , entonces

las **quintuplas** que definen δ **constituyen en sí un algoritmo**, pues son una lista finita de instrucciones, codificadas en un lenguaje, y que pueden llevarse a cabo de manera mecánica.

Alonzo Church enunció una tesis similar, pero utilizando otro formalismo: las funciones recursivas parciales.

Suele mencionarse este conjunto de tesis como la **Tesis de Turing-Church**.

La Tesis de Turing-Church

Tesis de Turing

Todo proceso que corresponda a un **procedimiento efectivo** puede ser realizado por una **Máquina de Turing**, es decir es Turing-computable.

Tesis de Church

Todo proceso que corresponda a un **procedimiento efectivo** o las fcs. efectivamente computables constituyen la clase de **Funciones Recursivas Parciales**.

La Tesis de Turing-Church - implicancias



Programa P



MT T equivalente a P

Dado que son equivalentes, para estudiar el poder de cómputo de las computadoras, podemos estudiar las Máquinas de Turing, cuyo funcionamiento es más simple.

Justificando la Tesis Turing-Church

Además de la Máquina de Turing, otros modelos han demostrado ser equivalentes:

- 1) Funciones Recursivas Parciales (Gödel, 1928-Kleene, 1935)
- 2) λ -cálculo de Church (1936)
- 3) Sistemas canónicos de Post (1943)
- 4) Algoritmo Generalizado de Markov (1951)
- 5) Gram. Estructuradas por frases (Chomsky, 1959-63)
- 6) Redes de Petri (1962)
- 7) Máq. de Registros de Shepardson y Sturgis (1963)

Algunos enfoques

- 1) Las caracterizaciones de Turing, Markov, Kleene, Church, Post y otros han demostrado ser **equivalentes**.
- 2) Se ha estudiado una gran variedad de fc. parciales intuitivamente algoritmizables. Para cada una se ha demostrado que es representable en alguna de las caracterizaciones formales anteriores.

Importante: la Tesis de Turing se acepta –en última instancia- con una base empírica.

Implicancias de la Tesis Turing-Church

Consideremos la siguiente implicancia de la Tesis de Turing:

Si existe un procedimiento efectivo para efectuar una manipulación de símbolos, entonces existe una Máquina de Turing que puede realizarlo

Cuando definimos una MT, utilizamos ciertos símbolos especiales, como $S_0, S_1, a, b, \dots, \delta$, etc.

Cuando nosotros “ejecutamos” una Máquina de Turing, haciendo una especie de traza...

¿no estamos, de alguna manera, manipulando símbolos?

Máquina Universal de Turing

Pensemos...

- Al ejecutar una máquina de Turing, simulamos acciones codificadas en la fc. δ
- El proceso de **mirar** un símbolo en la cinta, el estado actual, y **ver** si hay alguna quintupla ejecutable, y **realizar las modificaciones** que indica, definen un **procedimiento efectivo**.
- Por tesis de Turing-Church, si existe un procedimiento efectivo, existe una MT...

¡Entonces podemos construir una Máquina de Turing que “ejecute” otras Máquinas de Turing!

Máquina Universal de Turing

Esto es lo que se denomina Máquina de Propósito General o Máquina Universal de Turing, pues no fue construida para un problema específico.

Descripción de una MT T

Cadena de entrada α



Ejecución de T sobre α

Máquina Universal de Turing

Esto es lo que se denomina Máquina de Propósito General o Máquina Universal de Turing, pues no fue construida para un problema específico.



Un resultado notable y que cambió la Historia

Programa y Datos pueden almacenarse en un mismo dispositivo (cinta), usando el mismo tipo de símbolos (alfabeto)... este concepto tiene consecuencias en todas las “máquinas digitales” que usamos hoy.



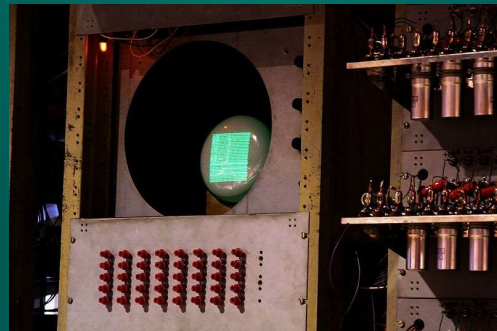
Un poco de historia...

- En 1945, el físico e ingeniero John Von Neumann redacta el primer borrador de lo que luego sería la EDVAC. Idea: construir una computadora electrónica basada en la Máquina Universal de Turing (MUT), con la noción de “programa almacenado”.
- En 1948, Frederic C. Williams, Tom Kilburn and Geoff Tootill construyen la Manchester Small-Scale Experimental Machine (SSEM) en la Univ. de Manchester (Inglaterra), apodada “Baby Machine”. Es la primer computadora que tiene un programa almacenado. Podía guardar 32 instrucciones. El 21 de junio de 1948 se ejecuta por primera vez un programa almacenado.
- La máquina está reconstruida en el Museum of Science & Engineering de Manchester, y puede verse en funcionamiento todos los martes.

Manchester Small-Scale Experimental Machine



Output: un tubo de rayos catódicos



Un poco de historia...

Almacenamiento de la SSEM: 32 números y 32 instrucciones.

El primer programa almacenado tenía 17 instrucciones. Escrito por Tom Kilburn, fue diseñado para encontrar el factor más grande de 2^{18} probando todos los números desde $2^{18} - 1$ hacia abajo.

Ejecutarlo llevó 3,5 millones de operaciones, y tardó 52 minutos.



Tom Kilburn
(1921-2001)

En honor a Turing...

- Existe la Turing Award (Premio Turing) de la ACM (Association for Computing Machinery)
- Existe el lenguaje de programación Turing, desarrollado en el Imperial College, Reino Unido.
- Existió un grupo de rock instrumental llamado “The Turing Machine”, fundado en 1998...
- Año 2012: se cumplen 100 años del nacimiento de A.Turing



- Más información:
<http://www.turing.org.uk/>



¿Cómo definir una MUT?

Si queremos definir una **MUT U**, para simular una Máq. de Turing **T** cualquiera, con una cadena α dada, las quintuplas de **T** deben **codificarse** de alguna manera

Ejemplo:

- I (izq.) representado por 1
- D (der.) representado por 11
- N (nada) representado por 111
- * separa símbolos
- ** separa quintuplas de T
- *** separa quintuplas de T y α

¿Cómo definir una MUT?

Consideremos p.ej. una MT con reglas $\delta(S_0, 0) = (S_1, 1, D)$ y $\delta(S_0, 1) = (S_1, 1, I)$, y sea $\alpha = 01$.



Est. Act.	Sim.Act.	Prox.Est.	Sim.Escrito	Mov.
0	0	1	1	D (= 11)
0	1	1	1	I (=1)



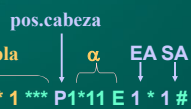
1er.quintupla 2da.quintupla cadena α

1 * 1 * 11 * 11 * 11 ** 1 * 11 * 11 * 11 * 1 *** 1*11

¿Cómo definir una MUT?

La MUT para simular a **T** necesitará identificar

- dónde está la **cabeza de T** (notado con P)
- cuál es el **espacio de trabajo (E)**, dónde se registrará cuál es **estado actual (EA)** y cuál es el **símbolo actualmente leído (SA)**.



1er.quintupla 2da.quintupla pos.cabeza α EA SA

1 * 1 * 11 * 11 * 11 ** 1 * 11 * 11 * 11 * 1 *** P1*11 E 1 * 1

La MUT U llevará a cabo su actividad en ciclos. Cada ciclo es una computación de T sobre α .

Una MUT en ejecución

Repetir

Buscar en quintuplas de T, comparando sus dos primeros símbolos con los del espacio de trabajo de U.

Si no hay tales símbolos entonces parar

sino

Seleccionar quintupla.

Cambiar el símbolo sgte. a P por el que aparece en 4to. lugar en la quintupla (escribir).

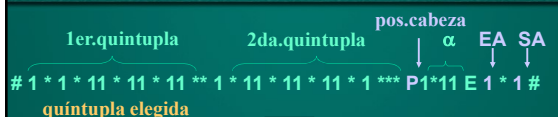
Cambiar el símbolo sgte. a E por el que aparece en 3er. lugar en la quintupla (cambiar estado).

Mover P a izquierda o derecha según lo indicado por el 5to. lugar en la quintupla.

Copiar el símbolo que sigue a P en el 2do. bloque de E, para mostrar cuál es el nuevo símbolo que T está leyendo.

Hasta parar

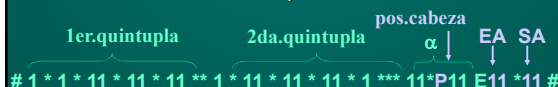
Una MUT en ejecución: ejemplo



1er.quintupla 2da.quintupla pos.cabeza α EA SA

1 * 1 * 11 * 11 * 11 ** 1 * 11 * 11 * 11 * 1 *** P1*11 E 1 * 1

quintupla elegida



1er.quintupla 2da.quintupla pos.cabeza α EA SA

1 * 1 * 11 * 11 * 11 ** 1 * 11 * 11 * 11 * 1 *** 11*P11 E11 *11

STOP

Limitaciones de Computabilidad

- Sabemos qué es una máquina de Turing, y qué cosas puede hacer...
- La Tesis de Turing-Church interrelaciona la noción de procedimiento efectivo y máquina de Turing.
- Vale preguntarse ¿qué cosas las máquinas de Turing NO pueden hacer?

Podemos categorizar a los problemas de acuerdo a la posibilidad de resolverlos en términos de máquinas de Turing

Problemas de Decisión

Def.: un **problema de decisión (PD)** es aquel formulado por una pregunta (referida a alguna propiedad) que requiere una respuesta de tipo “sí/no”.

Ej: problemas de pertenencia (o bien $\in$ o bien $\notin$)



Problemas de Decisión

Un problema de decisión es

- **soluble** si existe un **algoritmo total** para determinar si la propiedad es verdadera ($\Leftrightarrow$ existe una MT que siempre para al resolver el problema)
- **parcialmente soluble** si existe un **algoritmo parcial** para determinar si la propiedad es verdadera ($\Leftrightarrow$ existe una MT que resuelve el problema, pero puede no parar!)
- **insoluble** si **no existe un procedimiento efectivo** para determinar si la propiedad es verdadera ($\Leftrightarrow$ no existe una MT)

Ejemplos de Problemas de Decisión

- ¿Existe un algoritmo para decidir si un número natural cualquiera es par?
Sí $\Rightarrow$ es soluble.
- ¿Existe un algoritmo para decidir si dos autómatas finitos cualesquiera son equivalentes?
Sí $\Rightarrow$ es un problema soluble.
- ¿Existe un algoritmo para determinar si una gramática es ambigua o no?
No $\Rightarrow$ es insoluble.
- Muchos problemas insolubles son paradójicos en su naturaleza. Ej: la paradoja de Russell

Un peluquero afeita a todas las personas que no se afeitan a sí mismas. El peluquero: ¿se afeita a sí mismo? (Conocida como “paradoja del peluquero” - insoluble)



Problema de la Detención de MT

Un problema de decisión de especial importancia es el “**problema de detención de la máquina de Turing**” (**Halting Problem**)

Problema de Detención de MT (The Halting Problem): Dada una MT T y una cadena α , ¿existe un algoritmo para decidir si T se detendrá comenzando en el estado inicial con α en la cinta?

¡Problema Insoluble!

Problema de la Detención de MT



¡Puedo demostrar que este algoritmo no existe!



Algoritmo DetenciónMT

Dato de Entrada: T, α

Dato de Salida: Estado

Comienzo

.....

Devolver Estado con valor

“se detiene” o “cicla”

Fin

Problema de la Detención de MT

Alan Turing, a fines de 1930, enunció y demostró que este problema es insoluble.

Teorema: El problema de la detención de la máquina de Turing no es algorítmicamente soluble.

Veremos la demostración más adelante

Problema de la Detención de MT

.... # α #

MT T

¡Puedo demostrar que este algoritmo no existe!



Algoritmo DetenciónMT
Dato de Entrada: T, α
Dato de Salida: Estado
Comienzo

.....
Devolver Estado con valor “se detiene” o “cicla”
Fin

Turing ha demostrado que este algoritmo NO EXISTE

Problema de la Detención de MT

En un lenguaje de alto nivel (ej: Pascal) podríamos escribir un simulador de máquinas de Turing. Dada una MT T y una cadena α , podrán ocurrir tres situaciones

1. El cómputo de T es exitoso; el simulador dará como salida el resultado de la evaluación y se detendrá.
2. T no puede computar el valor para α , y se detendrá (ej: imprimiendo un mensaje).
3. T no se detiene y cicla. El simulador también lo hará. ¡He aquí donde el problema de la detención se hace presente!

Un Simulador de programas...

- Ya hemos visto simuladores de “máquinas de Turing” antes... Cuando usamos un debugger.
- El problema de la detención nos dice que jamás podremos escribir un “debugger” que tenga una función especial “verificar algoritmo” y que nos diga si el mismo se comporta correctamente o no.
- Este es un problema central en las Ciencias de la Computación.
- Por eso se estudian técnicas de testeo de programas, como un paliativo ante esta imposibilidad.